



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 1-2
Task : Project I
Due : 21st November 2025 (10%)

Name	Matric No.
CHAU YING JIA	A23CS0213
TAN ZHI MING	A23CS0189
NG YU HIN	A23CS0148
ELIJAH SHE YU SHENG	A23CS0073

Instructions

Each group must submit **one (1)** project only.

Your submission must include:

1. **Project Report in PDF format** (named as LeaderName_GroupNum_ProjectI.pdf).
2. The **demo video URL** (YouTube or Google Drive link) **inside the report**.
 - Make sure the link is **working, publicly accessible**, and does **not require login**.
3. Include all SQL script files (.sql) by attaching them directly, compressing them into one zip folder, or providing a valid OneDrive sharing link inside the report.

Introduction

Our group choose to develop an attendance system for university and schools. Almost all of the university and school need to take student attendance every day as it plays an important role in academic performance, discipline monitoring, and institutional reporting. The traditional methods like using paper-sign sheets are slow and prone to errors. Sometimes, it is hard for lecturers or student to find old attendance records. In this project, we wanted to solve these issues by building the core database for a modern and digitalize attendance system. This system will allow lecturers to record students' attendance efficiently and accurately to reduce mistakes. We wanted to create a well-designed and centralized database for managing student, course, lecturers and class section information in an attendance management system.

The objective of this project is to develop a complete SQL-based backend capable of supporting the key operations of an attendance system. This includes designing stable table structures using DDL, inserting and managing data using DML, and retrieving meaningful information using advanced SQL queries.

Our system aims to:

1. Provide a structured database to store student, lecturer, course, enrolment, session, and attendance data consistently.
2. Ensure accuracy and data integrity using constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and DEFAULT.
3. Improve system performance through indexing and query optimization.
4. Demonstrate backend logic that can later be integrated with any front-end application for real-world use.

This project is completed collaboratively by our group, with each member responsible for different parts of the SQL backend:

1. Chau Ying Jia (A23CS0213) – Handled DDL part and some report writing.
2. Tan Zhi Ming (A23CS0189) – Worked on DML operations and proceed to the DQL.
3. Ng Yu Hin (A23CS0148) – Focused on the remaining DQL parts.
4. Elijah She Yu Sheng (A23CS0073) – Managed the indexing and analysing part.

A. Database Creation (DDL)

- Create database and tables using CREATE DATABASE and CREATE TABLE.
- Apply constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT.

Code	Output
CREATE DATABASE	• <code>CREATE DATABASE AttendanceSystemDB;</code> • <code>USE AttendanceSystemDB;</code> Create a database named “AttendanceSystemDB” and use the database for this project.
CREATE TABLE	• <code>CREATE TABLE Students (</code> <code> StudentID VARCHAR(10) PRIMARY KEY,</code> <code> SFirstName VARCHAR(50) NOT NULL,</code> <code> SLastName VARCHAR(50) NOT NULL,</code> <code> SPhone VARCHAR(20),</code> <code> EnrollmentYear INT CHECK (EnrollmentYear >= 2000),</code> <code> Status VARCHAR(10) DEFAULT 'Active'</code> <code> CHECK (Status IN ('Active', 'Inactive'))</code> <code>);</code> Create a table of Students storing students’ information. Use of PRIMARY KEY, NOT NULL, CHECK and DEFAULT. For CHECK key, it ensures the value of insert meets the condition before inserting. • <code>CREATE TABLE Lecturers (</code> <code> LecturerID INT PRIMARY KEY AUTO_INCREMENT,</code> <code> LFirstName VARCHAR(50) NOT NULL,</code> <code> LLastName VARCHAR(50) NOT NULL,</code> <code> LEmail VARCHAR(100) UNIQUE,</code> <code> Department VARCHAR(100)</code> <code>);</code> Create a table of Lecturers storing lecturers information. Use of PRIMARY KEY, NOT NULL and UNIQUE. For UNIQUE, it ensures there are no students who take attendance twice. • <code>CREATE TABLE Courses (</code> <code> CourseID VARCHAR(15) PRIMARY KEY,</code> <code> CourseName VARCHAR(100) NOT NULL,</code> <code> Credits INT CHECK (Credits BETWEEN 1 AND 6),</code> <code> Semester VARCHAR(20),</code> <code>-- e.g. '2025S1'</code> <code> Section VARCHAR(10) NOT NULL,</code> <code> LecturerID INT NOT NULL,</code> <code> FOREIGN KEY (LecturerID) REFERENCES Lecturers(LecturerID)</code> <code>);</code> Create a table of Courses storing courses information. Use of PRIMARY KEY, NOT NULL and CHECK.

	 <pre> CREATE TABLE Enrollments (EnrollmentID INT PRIMARY KEY AUTO_INCREMENT, StudentID VARCHAR(10) NOT NULL, CourseID VARCHAR(15) NOT NULL, EnrollmentDate DATE DEFAULT (CURRENT_DATE), Status VARCHAR(10) DEFAULT 'Enrolled' CHECK (Status IN ('Enrolled', 'Dropped')), FOREIGN KEY (StudentID) REFERENCES Students(StudentID), FOREIGN KEY (CourseID) REFERENCES Courses(CourseID), UNIQUE (StudentID, CourseID)); </pre> Create a table of Enrollments storing student enrollment information. Use of PRIMARY KEY, NOT NULL, DEFAULT and CHECK.  <pre> CREATE TABLE AttendanceRecords (AttendanceID INT PRIMARY KEY AUTO_INCREMENT, StudentID VARCHAR(10) NOT NULL, SessionID INT NOT NULL, Status VARCHAR(10) NOT NULL CHECK (Status IN ('Present', 'Absent', 'Late', 'Excused')), TimeRecorded DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (StudentID) REFERENCES Students(StudentID), FOREIGN KEY (SessionID) REFERENCES Sessions(SessionID), UNIQUE (StudentID, SessionID)); </pre> Create a table of AttendanceRecords storing attendance information. Use of PRIMARY KEY, CHECK, UNIQUE AND NOT NULL.  <pre> CREATE TABLE Temp_CourseTable (CourseCode VARCHAR(10) PRIMARY KEY, Section INT); </pre> Create a temporary table.
--	--

- Include ALTER TABLE and DROP TABLE statement.

Code	Output
ALTER TABLE	 <pre> ALTER TABLE Courses MODIFY Section VARCHAR(5) NOT NULL; </pre>  <pre> ALTER TABLE Students ADD SEmail VARCHAR(100) UNIQUE; </pre>

	Alter Courses table by modifying the section datatype and add the student email column to Students table.
DROP TABLE	DROP TABLE Temp_CourseTable; Drop the temporary table.

B. Data Manipulation (DML)

- Insert at least 10 records per table using INSERT.

Task	Code
INSERT values to table Lecturers	<pre>INSERT INTO Lecturers (LFirstName, LLastName, LEmail, Department) VALUES ('Ahmad', 'Rahim', 'ahmadrahim@utm.my', 'Languages'), ('Siti', 'Zaharah', 'sitizaharah@utm.m', 'Languages'), ('Lim', 'Wei Ming', 'limweiming@utm.m', 'Computer Science'), ('Tan', 'Jun Hao', 'tanjunhao@uutm.m', 'Computer Science'), ('Aisyah', 'Hamzah', 'aisyahhamzah@utm.m', 'Mathematics'), ('Gurmeet', 'Singh', 'gurmeetsingh@utm.m', 'Mathematics'), ('Farah', 'Husna', 'farahhusna@utm.m', 'Business'), ('Wong', 'Kai Lun', 'wongkailun@utm.m', 'Business'), ('Saravanan', 'Maniam', 'saravananmaniam@utm.m', 'Engineering'), ('Nurul', 'Aqilah', 'nurulaqilah@utm.m', 'Engineering');</pre>
INSERT values to table Students	<pre>INSERT INTO Students (StudentID, SFirstName, SLastName, SPhone, EnrollmentYear, Status, SEmail) VALUES ('S001', 'Tan', 'Zhiming', '012-1111111', 2023, 'Active', 'tangzhiming@graduate.utm.my'), ('S002', 'Nadia', 'Syafiqah', '012-2222222', 2022, 'Active', 'nadiasyafiqah@graduate.utm.my'), ('S003', 'Wei', 'Jie', '012-3333333', 2024, 'Active', 'weijie@graduate.utm.my'), ('S004', 'Chau', 'Yingjie', '012-4444444', 2021, 'Active', 'chauyingjie@graduate.utm.my'), ('S005', 'Hafiz', 'Muzakir', '012-5555555', 2023, 'Active', 'hafizmuzakir@graduate.utm.my'), ('S006', 'Ng', 'Yuhin', '012-6666666', 2022, 'Active', 'ngyuhin@graduate.utm.my'), ('S007', 'Aisyan', 'Habila', '012-7777777', 2023, 'Active', 'aisyahnabila@graduate.utm.my'), ('S008', 'Yoges', 'Kumar', '012-8888888', 2024, 'Active', 'yogeskumar@graduate.utm.my'), ('S009', 'Elijah', 'She', '012-9999999', 2021, 'Active', 'elijahshe@graduate.utm.my'), ('S010', 'Firdaus', 'Rashid', '012-0000000', 2020, 'Active', 'firdausrashid@graduate.utm.my');</pre>
INSERT values to table Courses	<pre>INSERT INTO Courses (CourseID, CourseName, Credits, Semester, Section, LecturerID) VALUES ('FRN101', 'Bahasa Perancis I', 3, '2025S1', 'A1', 1), ('FRN201', 'Bahasa Perancis II', 3, '2025S1', 'B1', 2), ('CSC101', 'Asas Pengaturcaraan', 4, '2025S1', 'A1', 3), ('CSC201', 'Sistem Pangkalan Data', 4, '2025S1', 'B1', 4), ('MAT101', 'Kalkulus I', 3, '2025S1', 'A1', 5), ('MAT201', 'Algebra Linear', 3, '2025S1', 'B1', 6), ('BUS101', 'Prinsip Perniagaan', 3, '2025S1', 'A1', 7), ('BUS201', 'Asas Pemasaran', 3, '2025S1', 'B1', 8), ('ENG101', 'Mekanikal Kejuruteraan', 4, '2025S1', 'A1', 9), ('ENG201', 'Termodinamik', 4, '2025S1', 'B1', 10);</pre>
INSERT values to table Enrollments	<pre>INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate, Status) VALUES ('S001', 'FRN101', '2025-02-01', 'Enrolled'), ('S002', 'FRN101', '2025-02-01', 'Enrolled'), ('S003', 'FRN101', '2025-02-02', 'Enrolled'), ('S004', 'CSC201', '2025-02-01', 'Enrolled'), ('S005', 'CSC201', '2025-02-01', 'Enrolled'), ('S006', 'MAT101', '2025-02-01', 'Enrolled'), ('S007', 'MAT101', '2025-02-03', 'Enrolled'), ('S008', 'BUS101', '2025-02-01', 'Enrolled'), ('S009', 'ENG101', '2025-02-02', 'Enrolled'), ('S010', 'ENG101', '2025-02-02', 'Enrolled');</pre>
INSERT values to table Sessions	<pre>INSERT INTO Sessions (CourseID, SessionDate) VALUES ('FRN101', '2025-03-03 08:00:00'), ('FRN101', '2025-03-10 08:00:00'), ('FRN101', '2025-03-17 08:00:00'), ('CSC201', '2025-03-03 10:00:00'), ('CSC201', '2025-03-10 10:00:00'), ('MAT101', '2025-03-04 09:00:00'), ('MAT101', '2025-03-11 09:00:00'), ('BUS101', '2025-03-05 14:00:00'), ('ENG101', '2025-03-06 11:00:00'), ('ENG101', '2025-03-13 11:00:00');</pre>

Insert values to table AttendanceRecords	<pre> INSERT INTO AttendanceRecords (StudentID, SessionID, Status) VALUES ('S001', 1, 'Present'), ('S002', 1, 'Absent'), ('S003', 1, 'Late'), ('S001', 2, 'Present'), ('S002', 2, 'Present'), ('S003', 2, 'Excused'), ('S004', 4, 'Present'), ('S005', 4, 'Absent'), ('S006', 6, 'Present'), ('S007', 7, 'Late'); </pre>
--	--

- Perform updates and deletions with appropriate conditions (UPDATE, DELETE with WHERE).

Task	Code
Update : 1. Set student status to inactive if enrollment year <= 2021 2. Drop student (Hafiz Muzakir) enrollment from course (Sistem Pangkalan Data) 3. Update lecturer ahmad rahim email	<pre> -- set student status to inactive if enrollment year <= 2021 -- UPDATE Students SET Status = 'Inactive' WHERE EnrollmentYear <= 2021; -- Drop student (Hafiz Muzakir) enrollment from course (Sistem Pangkalan Data) -- UPDATE Enrollments SET Status = 'Dropped' WHERE StudentID = 'S005' AND CourseID = 'CSC201'; -- update lecturer ahmad rahim email -- UPDATE Lecturers SET LEmail = 'ahmad.rahim@utm.my' WHERE LFirstName = 'Ahmad' AND LLastName = 'Rahim'; </pre>
Delete: 1. Delete student with excused from attendance records 2. Remove Prinsip Perniagaan from session at 5/3/2025 2pm 3. Remove student firdaus enrollment for course Mekanikal Kejuruteraan from table enrollments	<pre> -- delete student with excused from attendance records -- DELETE FROM AttendanceRecords WHERE Status = 'Excused'; -- remove Prinsip Perniagaan from session at 5/3/2025 2pm -- DELETE FROM Sessions WHERE CourseID = 'BUS101' AND SessionDate = '2025-03-05 14:00:00'; -- Remove student firdaus enrollment for course Mekanikal Kejuruteraan from table enrollments -- DELETE FROM Enrollments WHERE StudentID = 'S0010' AND CourseID = 'ENG101'; </pre>

C. Data Retrieval (DQL / SELECT)

Develop at least **following SQL queries** that demonstrate:

1. Filtering (WHERE, AND, OR, NOT, BETWEEN, LIKE, NULL).

Code	Output																																				
Filter student who is active and enrollment year between 2021 - 2024 <pre>-- filtering -- SELECT StudentID, SFirstName, SLastName, EnrollmentYear FROM Students WHERE EnrollmentYear BETWEEN 2021 AND 2024 AND Status = 'Active';</pre>	<table><tr><th>StudentID</th><th>SFirstName</th><th>SLastName</th><th>EnrollmentYear</th></tr><tr><td>S001</td><td>Tan</td><td>Zhiming</td><td>2023</td></tr><tr><td>S002</td><td>Nadia</td><td>Syafiqah</td><td>2022</td></tr><tr><td>S003</td><td>Wei</td><td>Jie</td><td>2024</td></tr><tr><td>S005</td><td>Hafiz</td><td>Muzakir</td><td>2023</td></tr><tr><td>S006</td><td>Ng</td><td>Yuhin</td><td>2022</td></tr><tr><td>S007</td><td>Aisyah</td><td>Nabila</td><td>2023</td></tr><tr><td>S008</td><td>Yoges</td><td>Kumar</td><td>2024</td></tr><tr><td>NULL</td><td>NULL</td><td>NULL</td><td>NULL</td></tr></table>	StudentID	SFirstName	SLastName	EnrollmentYear	S001	Tan	Zhiming	2023	S002	Nadia	Syafiqah	2022	S003	Wei	Jie	2024	S005	Hafiz	Muzakir	2023	S006	Ng	Yuhin	2022	S007	Aisyah	Nabila	2023	S008	Yoges	Kumar	2024	NULL	NULL	NULL	NULL
StudentID	SFirstName	SLastName	EnrollmentYear																																		
S001	Tan	Zhiming	2023																																		
S002	Nadia	Syafiqah	2022																																		
S003	Wei	Jie	2024																																		
S005	Hafiz	Muzakir	2023																																		
S006	Ng	Yuhin	2022																																		
S007	Aisyah	Nabila	2023																																		
S008	Yoges	Kumar	2024																																		
NULL	NULL	NULL	NULL																																		
Filter course that includes word ‘Bahasa’ / ‘Pemasaran’ <pre>SELECT CourseID, CourseName FROM Courses WHERE CourseName LIKE '%Bahasa%' OR CourseName LIKE '%Pemasaran%';</pre>	<table><tr><th></th><th>CourseID</th><th>CourseName</th></tr><tr><td>▶</td><td>BUS201</td><td>Asas Pemasaran</td></tr><tr><td></td><td>FRN101</td><td>Bahasa Perancis I</td></tr><tr><td></td><td>FRN201</td><td>Bahasa Perancis II</td></tr><tr><td>✱</td><td>NULL</td><td>NULL</td></tr></table>		CourseID	CourseName	▶	BUS201	Asas Pemasaran		FRN101	Bahasa Perancis I		FRN201	Bahasa Perancis II	✱	NULL	NULL																					
	CourseID	CourseName																																			
▶	BUS201	Asas Pemasaran																																			
	FRN101	Bahasa Perancis I																																			
	FRN201	Bahasa Perancis II																																			
✱	NULL	NULL																																			
Filter lecturer from mathematics departments with not null in attribute Email <pre>SELECT LFirstName, LLastName, LEmail FROM Lecturers WHERE Department = 'Mathematics' AND LEmail IS NOT NULL;</pre>	Result Grid Filter Rows: <table><tr><th></th><th>LFirstName</th><th>LLastName</th><th>LEmail</th></tr><tr><td>▶</td><td>Aisyah</td><td>Hamzah</td><td>aisyahhamzah@utm.m</td></tr><tr><td></td><td>Gurmeet</td><td>Singh</td><td>gurmeetsingh@utm.m</td></tr></table>		LFirstName	LLastName	LEmail	▶	Aisyah	Hamzah	aisyahhamzah@utm.m		Gurmeet	Singh	gurmeetsingh@utm.m																								
	LFirstName	LLastName	LEmail																																		
▶	Aisyah	Hamzah	aisyahhamzah@utm.m																																		
	Gurmeet	Singh	gurmeetsingh@utm.m																																		

2. Sorting (ORDER BY, LIMIT).

Code	Output																																												
Sort student by enrollment years with ascending order																																													
<pre>-- sorting -- SELECT StudentID, SFirstName, EnrollmentYear FROM Students ORDER BY EnrollmentYear ASC;</pre>	<table><tr><th></th><th>StudentID</th><th>SFirstName</th><th>EnrollmentYear</th></tr><tr><td>▶</td><td>S010</td><td>Firdaus</td><td>2020</td></tr><tr><td></td><td>S004</td><td>Chau</td><td>2021</td></tr><tr><td></td><td>S009</td><td>Elijah</td><td>2021</td></tr><tr><td></td><td>S002</td><td>Nadia</td><td>2022</td></tr><tr><td></td><td>S006</td><td>Ng</td><td>2022</td></tr><tr><td></td><td>S001</td><td>Tan</td><td>2023</td></tr><tr><td></td><td>S005</td><td>Hafiz</td><td>2023</td></tr><tr><td></td><td>S007</td><td>Aisyah</td><td>2023</td></tr><tr><td></td><td>S003</td><td>Wei</td><td>2024</td></tr><tr><td></td><td>S008</td><td>Yoges</td><td>2024</td></tr></table>		StudentID	SFirstName	EnrollmentYear	▶	S010	Firdaus	2020		S004	Chau	2021		S009	Elijah	2021		S002	Nadia	2022		S006	Ng	2022		S001	Tan	2023		S005	Hafiz	2023		S007	Aisyah	2023		S003	Wei	2024		S008	Yoges	2024
	StudentID	SFirstName	EnrollmentYear																																										
▶	S010	Firdaus	2020																																										
	S004	Chau	2021																																										
	S009	Elijah	2021																																										
	S002	Nadia	2022																																										
	S006	Ng	2022																																										
	S001	Tan	2023																																										
	S005	Hafiz	2023																																										
	S007	Aisyah	2023																																										
	S003	Wei	2024																																										
	S008	Yoges	2024																																										

Sort course by credits with descending order (only show up to 5 , LIMIT)

```
SELECT CourseID, CourseName, Credits
FROM Courses
ORDER BY Credits DESC
LIMIT 5;
```

	CourseID	CourseName	Credits
▶	CSC101	Asas Pengaturcaraan	4
	CSC201	Sistem Pangkalan Data	4
	ENG101	Mekanikal Kejuruteraan	4
	ENG201	Termodinamik	4
	BUS101	Prinsip Perniagaan	3
*	NULL	NULL	NULL

3. Aggregation (COUNT, SUM, AVG, MAX, MIN).

Code	Output																				
Count total students <pre>-- aggregation -- SELECT COUNT(*) AS TotalStudents FROM Students;</pre>	<table><tr><th>TotalStudents</th></tr><tr><td>10</td></tr></table>	TotalStudents	10																		
TotalStudents																					
10																					
Show average, min, max course credit. <pre>SELECT AVG(Credits) AS AvgCredits, MAX(Credits) AS MaxCredits, MIN(Credits) AS MinCredits FROM Courses;</pre>	<table><tr><th>AvgCredits</th><th>MaxCredits</th><th>MinCredits</th></tr><tr><td>3.4000</td><td>4</td><td>3</td></tr></table>	AvgCredits	MaxCredits	MinCredits	3.4000	4	3														
AvgCredits	MaxCredits	MinCredits																			
3.4000	4	3																			
<pre>SELECT e.StudentID, SUM(c.Credits) AS TotalCredits FROM Enrollments e JOIN Courses c ON e.CourseID = c.CourseID GROUP BY e.StudentID;</pre>	<table><tr><th>StudentID</th><th>TotalCredits</th></tr><tr><td>S001</td><td>3</td></tr><tr><td>S002</td><td>3</td></tr><tr><td>S003</td><td>3</td></tr><tr><td>S004</td><td>4</td></tr><tr><td>S005</td><td>4</td></tr><tr><td>S006</td><td>3</td></tr><tr><td>S007</td><td>3</td></tr><tr><td>S008</td><td>3</td></tr><tr><td>S009</td><td>4</td></tr></table>	StudentID	TotalCredits	S001	3	S002	3	S003	3	S004	4	S005	4	S006	3	S007	3	S008	3	S009	4
StudentID	TotalCredits																				
S001	3																				
S002	3																				
S003	3																				
S004	4																				
S005	4																				
S006	3																				
S007	3																				
S008	3																				
S009	4																				

4. Grouping and filtering groups (GROUP BY, HAVING).

Code	Output																				
Show total sessions of grouped course (only show course that have session 2 and above) <pre>-- grouping -- SELECT CourseID, COUNT(*) AS TotalSessions FROM Sessions GROUP BY CourseID HAVING COUNT(*) >= 2;</pre>	<table><tr><th></th><th>CourseID</th><th>TotalSessions</th></tr><tr><td>▶</td><td>CSC201</td><td>2</td></tr><tr><td></td><td>ENG101</td><td>2</td></tr><tr><td></td><td>FRN101</td><td>3</td></tr><tr><td></td><td>MAT101</td><td>2</td></tr></table>		CourseID	TotalSessions	▶	CSC201	2		ENG101	2		FRN101	3		MAT101	2					
	CourseID	TotalSessions																			
▶	CSC201	2																			
	ENG101	2																			
	FRN101	3																			
	MAT101	2																			
Show student attendance rate (only show student that had been at least attendance one) <pre>SELECT StudentID, COUNT(*) AS TotalRecords, SUM(Status = 'Present') AS PresentCount FROM AttendanceRecords GROUP BY StudentID HAVING SUM(Status = 'Present') >= 1;</pre>	<table><tr><th></th><th>StudentID</th><th>TotalRecords</th><th>PresentCount</th></tr><tr><td>▶</td><td>S001</td><td>2</td><td>2</td></tr><tr><td></td><td>S002</td><td>2</td><td>1</td></tr><tr><td></td><td>S004</td><td>1</td><td>1</td></tr><tr><td></td><td>S006</td><td>1</td><td>1</td></tr></table>		StudentID	TotalRecords	PresentCount	▶	S001	2	2		S002	2	1		S004	1	1		S006	1	1
	StudentID	TotalRecords	PresentCount																		
▶	S001	2	2																		
	S002	2	1																		
	S004	1	1																		
	S006	1	1																		

5. Numeric and string functions (ROUND, TRUNCATE, UPPER, LENGTH, CONCAT, SUBSTR).

Code	Output																																																							
Concat student name , and make the full name upper , and use substr to extract the first 3 letter of firstname. <pre>-- numeric and string function -- SELECT StudentID, UPPER(CONCAT(SFirstName, ' ', SLastName)) AS FullNameUpper, SUBSTR(SFirstName, 1, 3) AS FirstThreeLetters FROM Students;</pre>	<table><thead><tr><th></th><th>StudentID</th><th>FullNameUpper</th><th>FirstThreeLetters</th></tr></thead><tbody><tr><td>▶</td><td>S001</td><td>TAN ZHIMING</td><td>Tan</td></tr><tr><td></td><td>S002</td><td>NADIA SYAFIQAH</td><td>Nad</td></tr><tr><td></td><td>S003</td><td>WEI JIE</td><td>Wei</td></tr><tr><td></td><td>S004</td><td>CHAU YINGJIA</td><td>Cha</td></tr><tr><td></td><td>S005</td><td>HAFIZ MUZAKIR</td><td>Haf</td></tr><tr><td></td><td>S006</td><td>NG YUHIN</td><td>Ng</td></tr><tr><td></td><td>S007</td><td>AISYAH NABILA</td><td>Ais</td></tr><tr><td></td><td>S008</td><td>YOGES KUMAR</td><td>Yog</td></tr><tr><td></td><td>S009</td><td>ELIJAH SHE</td><td>Eli</td></tr><tr><td></td><td>S010</td><td>FIRDAUS RASHID</td><td>Fir</td></tr></tbody></table>		StudentID	FullNameUpper	FirstThreeLetters	▶	S001	TAN ZHIMING	Tan		S002	NADIA SYAFIQAH	Nad		S003	WEI JIE	Wei		S004	CHAU YINGJIA	Cha		S005	HAFIZ MUZAKIR	Haf		S006	NG YUHIN	Ng		S007	AISYAH NABILA	Ais		S008	YOGES KUMAR	Yog		S009	ELIJAH SHE	Eli		S010	FIRDAUS RASHID	Fir											
	StudentID	FullNameUpper	FirstThreeLetters																																																					
▶	S001	TAN ZHIMING	Tan																																																					
	S002	NADIA SYAFIQAH	Nad																																																					
	S003	WEI JIE	Wei																																																					
	S004	CHAU YINGJIA	Cha																																																					
	S005	HAFIZ MUZAKIR	Haf																																																					
	S006	NG YUHIN	Ng																																																					
	S007	AISYAH NABILA	Ais																																																					
	S008	YOGES KUMAR	Yog																																																					
	S009	ELIJAH SHE	Eli																																																					
	S010	FIRDAUS RASHID	Fir																																																					
Show rounded value and truncated value by course credit divide by 3 <pre>SELECT CourseID, Credits, ROUND(Credits / 3, 2) AS RoundedValue, TRUNCATE(Credits / 3, 1) AS TruncatedValue FROM Courses;</pre>	<table><thead><tr><th></th><th>CourseID</th><th>Credits</th><th>RoundedValue</th><th>TruncatedValue</th></tr></thead><tbody><tr><td>▶</td><td>BUS101</td><td>3</td><td>1.00</td><td>1.0</td></tr><tr><td></td><td>BUS201</td><td>3</td><td>1.00</td><td>1.0</td></tr><tr><td></td><td>CSC101</td><td>4</td><td>1.33</td><td>1.3</td></tr><tr><td></td><td>CSC201</td><td>4</td><td>1.33</td><td>1.3</td></tr><tr><td></td><td>ENG101</td><td>4</td><td>1.33</td><td>1.3</td></tr><tr><td></td><td>ENG201</td><td>4</td><td>1.33</td><td>1.3</td></tr><tr><td></td><td>FRN101</td><td>3</td><td>1.00</td><td>1.0</td></tr><tr><td></td><td>FRN201</td><td>3</td><td>1.00</td><td>1.0</td></tr><tr><td></td><td>MAT101</td><td>3</td><td>1.00</td><td>1.0</td></tr><tr><td></td><td>MAT201</td><td>3</td><td>1.00</td><td>1.0</td></tr></tbody></table>		CourseID	Credits	RoundedValue	TruncatedValue	▶	BUS101	3	1.00	1.0		BUS201	3	1.00	1.0		CSC101	4	1.33	1.3		CSC201	4	1.33	1.3		ENG101	4	1.33	1.3		ENG201	4	1.33	1.3		FRN101	3	1.00	1.0		FRN201	3	1.00	1.0		MAT101	3	1.00	1.0		MAT201	3	1.00	1.0
	CourseID	Credits	RoundedValue	TruncatedValue																																																				
▶	BUS101	3	1.00	1.0																																																				
	BUS201	3	1.00	1.0																																																				
	CSC101	4	1.33	1.3																																																				
	CSC201	4	1.33	1.3																																																				
	ENG101	4	1.33	1.3																																																				
	ENG201	4	1.33	1.3																																																				
	FRN101	3	1.00	1.0																																																				
	FRN201	3	1.00	1.0																																																				
	MAT101	3	1.00	1.0																																																				
	MAT201	3	1.00	1.0																																																				

Show student first name length

```
SELECT
    StudentID,
    SFirstName,
    LENGTH(SFirstName) AS FirstNameLength
FROM Students;
```

	StudentID	SFirstName	FirstNameLength
▶	S001	Tan	3
	S002	Nadia	5
	S003	Wei	3
	S004	Chau	4
	S005	Hafiz	5
	S006	Ng	2
	S007	Aisyah	6
	S008	Yoges	5
	S009	Elijah	6
	S010	Firdaus	7

6. Conditional logic (CASE WHEN).

Code	Output																																																								
Calculates attendance statistics (Present, Late, Absent) for each student and uses a CASE expression to categorize their overall attendance performance <pre>SELECT StudentID, COUNT(*) AS TotalSessions, SUM(Status = 'Present') AS present_count, SUM(Status = 'Late') AS late_count, SUM(Status = 'Absent') AS absent_count, CASE WHEN SUM(Status = 'Present') / COUNT(*) >= 0.8 THEN 'Good Attendance' WHEN SUM(Status = 'Absent') >= 2 THEN 'Poor Attendance' ELSE 'Average Attendance' END AS AttendanceStatus FROM AttendanceRecords GROUP BY StudentID;</pre>	Result Grid Filter Rows: Export: Wrap Cell Content: ☐ <table><thead><tr><th></th><th>StudentID</th><th>TotalSessions</th><th>present_count</th><th>late_count</th><th>absent_count</th><th>AttendanceStatus</th></tr></thead><tbody><tr><td>▶</td><td>S001</td><td>2</td><td>2</td><td>0</td><td>0</td><td>Good Attendance</td></tr><tr><td></td><td>S002</td><td>2</td><td>1</td><td>0</td><td>1</td><td>Average Attendance</td></tr><tr><td></td><td>S003</td><td>1</td><td>0</td><td>1</td><td>0</td><td>Average Attendance</td></tr><tr><td></td><td>S004</td><td>1</td><td>1</td><td>0</td><td>0</td><td>Good Attendance</td></tr><tr><td></td><td>S005</td><td>1</td><td>0</td><td>0</td><td>1</td><td>Average Attendance</td></tr><tr><td></td><td>S006</td><td>1</td><td>1</td><td>0</td><td>0</td><td>Good Attendance</td></tr><tr><td></td><td>S007</td><td>1</td><td>0</td><td>1</td><td>0</td><td>Average Attendance</td></tr></tbody></table>		StudentID	TotalSessions	present_count	late_count	absent_count	AttendanceStatus	▶	S001	2	2	0	0	Good Attendance		S002	2	1	0	1	Average Attendance		S003	1	0	1	0	Average Attendance		S004	1	1	0	0	Good Attendance		S005	1	0	0	1	Average Attendance		S006	1	1	0	0	Good Attendance		S007	1	0	1	0	Average Attendance
	StudentID	TotalSessions	present_count	late_count	absent_count	AttendanceStatus																																																			
▶	S001	2	2	0	0	Good Attendance																																																			
	S002	2	1	0	1	Average Attendance																																																			
	S003	1	0	1	0	Average Attendance																																																			
	S004	1	1	0	0	Good Attendance																																																			
	S005	1	0	0	1	Average Attendance																																																			
	S006	1	1	0	0	Good Attendance																																																			
	S007	1	0	1	0	Average Attendance																																																			

7. Subqueries (single-row, multiple-row, correlated).

Code	Output														
Single row subquery Find all students who enrolled on the earliest date in the system by comparing each record with a single-value subquery that returns the minimum enrollment date <pre>SELECT StudentID, CourseID FROM Enrollments WHERE EnrollmentDate = (SELECT MIN(EnrollmentDate) FROM Enrollments);</pre>	<table border="1"> <thead> <tr> <th>StudentID</th> <th>CourseID</th> </tr> </thead> <tbody> <tr><td>S001</td><td>FRN101</td></tr> <tr><td>S002</td><td>FRN101</td></tr> <tr><td>S004</td><td>CSC201</td></tr> <tr><td>S005</td><td>CSC201</td></tr> <tr><td>S006</td><td>MAT101</td></tr> <tr><td>S008</td><td>BUS101</td></tr> </tbody> </table>	StudentID	CourseID	S001	FRN101	S002	FRN101	S004	CSC201	S005	CSC201	S006	MAT101	S008	BUS101
StudentID	CourseID														
S001	FRN101														
S002	FRN101														
S004	CSC201														
S005	CSC201														
S006	MAT101														
S008	BUS101														

Code	Output
Multi row subquery	

Identifies lecturers who teach courses containing more than one session by checking whether their LecturerID appears in a grouped result set returned by the subquery

```
SELECT LecturerID, LFirstName, LLastName
FROM Lecturers
WHERE LecturerID IN (
    SELECT LecturerID
    FROM Courses c
    JOIN Sessions s ON c.CourseID = s.CourseID
    GROUP BY LecturerID
    HAVING COUNT(*) > 1
);
```

Result Grid			
	LecturerID	LFirstName	LLastName
▶	1	Ahmad	Rahim
	4	Tan	Jun Hao
	5	Aisyah	Hamzah
	9	Saravanan	Maniam
*	NULL	NULL	NULL

Code

Correlated subquery

Compares each student's total 'Present' count to the average 'Present' count of all students, where the outer query and inner aggregated subquery interact to produce a per-student comparison

```
SELECT StudentID
FROM AttendanceRecords a1
GROUP BY StudentID
HAVING SUM(Status='Present') >
(
    SELECT AVG(PresentCount)
    FROM (
        SELECT SUM(Status='Present') AS PresentCount
        FROM AttendanceRecords
        GROUP BY StudentID
    ) AS temp_table
);
```

Output	
Result Grid	
	StudentID
▶	S001
	S002
	S004
	S006

8. Set operations (UNION, NOT EXISTS).

Code	Output
------	--------

Union

Merges student names and lecturer names into a single unified result set using UNION, demonstrating set operations that combine compatible rows from different tables

```

SELECT S.FirstName AS FirstName, S.LastName AS LastName, 'Student' AS Role
FROM Students
UNION
SELECT L.FirstName, L.LastName, 'Lecturer'
FROM Lecturers;

```

Result Grid	Filter Rows:	
FirstName	LastName	Role
▶ Tan	Zhiming	Student
Nadia	Syafiqah	Student
Wei	Jie	Student
Chau	Yingjia	Student
Hafiz	Muzakir	Student
Ng	Rahim	Student
Aisyah	Nabila	Student
Yoges	Kumar	Student
Elijah	She	Student
Firdaus	Rashid	Student
Ahmad	Rahim	Lecturer
Siti	Zaharah	Lecturer
Lim	Wei Ming	Lecturer
Tan	Jun Hao	Lecturer
Aisyah	Hamzah	Lecturer
Gurmeet	Singh	Lecturer
Farah	Husna	Lecturer
Wong	Kai Lun	Lecturer
Saravanan	Maniam	Lecturer
Nurul	Aqilah	Lecturer

Code	Output									
NOT EXISTS Returns students who are enrolled in a course but do not yet have any attendance records by checking for the absence of matching SessionID records through NOT EXISTS <pre>SELECT e.StudentID, e.CourseID FROM Enrollments e WHERE NOT EXISTS (SELECT 1 FROM AttendanceRecords a WHERE a.StudentID = e.StudentID AND a.SessionID IN (SELECT SessionID FROM Sessions WHERE CourseID = e.CourseID))</pre>	<table><tr><th></th><th>StudentID</th><th>CourseID</th></tr><tr><td>▶</td><td>S008</td><td>BUS101</td></tr><tr><td></td><td>S009</td><td>ENG101</td></tr></table>		StudentID	CourseID	▶	S008	BUS101		S009	ENG101
	StudentID	CourseID								
▶	S008	BUS101								
	S009	ENG101								

9. Joins (NATURAL, INNER, LEFT OUTER, SELF).

Code	Output																																								
Natural Join Uses NATURAL JOIN to automatically match columns with the same names (like StudentID) between Students and Enrollments, producing combined enrollment data without specifying join conditions <pre>SELECT StudentID, CourseID, EnrollmentDate FROM (SELECT StudentID, CourseID, EnrollmentDate, Status AS EnrollStatus FROM Enrollments) AS e NATURAL JOIN Students;</pre>	Result Grid   Filter Rows: <table><thead><tr><th></th><th>StudentID</th><th>CourseID</th><th>EnrollmentDate</th></tr></thead><tbody><tr><td>▶</td><td>S001</td><td>FRN101</td><td>2025-02-01</td></tr><tr><td></td><td>S002</td><td>FRN101</td><td>2025-02-01</td></tr><tr><td></td><td>S003</td><td>FRN101</td><td>2025-02-02</td></tr><tr><td></td><td>S004</td><td>CSC201</td><td>2025-02-01</td></tr><tr><td></td><td>S005</td><td>CSC201</td><td>2025-02-01</td></tr><tr><td></td><td>S006</td><td>MAT101</td><td>2025-02-01</td></tr><tr><td></td><td>S007</td><td>MAT101</td><td>2025-02-03</td></tr><tr><td></td><td>S008</td><td>BUS101</td><td>2025-02-01</td></tr><tr><td></td><td>S009</td><td>ENG101</td><td>2025-02-02</td></tr></tbody></table>		StudentID	CourseID	EnrollmentDate	▶	S001	FRN101	2025-02-01		S002	FRN101	2025-02-01		S003	FRN101	2025-02-02		S004	CSC201	2025-02-01		S005	CSC201	2025-02-01		S006	MAT101	2025-02-01		S007	MAT101	2025-02-03		S008	BUS101	2025-02-01		S009	ENG101	2025-02-02
	StudentID	CourseID	EnrollmentDate																																						
▶	S001	FRN101	2025-02-01																																						
	S002	FRN101	2025-02-01																																						
	S003	FRN101	2025-02-02																																						
	S004	CSC201	2025-02-01																																						
	S005	CSC201	2025-02-01																																						
	S006	MAT101	2025-02-01																																						
	S007	MAT101	2025-02-03																																						
	S008	BUS101	2025-02-01																																						
	S009	ENG101	2025-02-02																																						

Code	Output																																																		
Inner Join Retrieves detailed enrollment information by joining Students, Enrollments, and Courses using explicit INNER JOIN conditions, showing only matched records across all involved tables <pre>SELECT e.StudentID, s.SFirstName, e.CourseID, c.CourseName FROM Enrollments e INNER JOIN Students s ON e.StudentID = s.StudentID INNER JOIN Courses c ON e.CourseID = c.CourseID;</pre>	<table><tr><th></th><th>StudentID</th><th>SFirstName</th><th>CourseID</th><th>CourseName</th></tr><tr><td>▶</td><td>S001</td><td>Tan</td><td>FRN101</td><td>Bahasa Perancis I</td></tr><tr><td></td><td>S002</td><td>Nadia</td><td>FRN101</td><td>Bahasa Perancis I</td></tr><tr><td></td><td>S003</td><td>Wei</td><td>FRN101</td><td>Bahasa Perancis I</td></tr><tr><td></td><td>S004</td><td>Chau</td><td>CSC201</td><td>Sistem Pangkalan Data</td></tr><tr><td></td><td>S005</td><td>Hafiz</td><td>CSC201</td><td>Sistem Pangkalan Data</td></tr><tr><td></td><td>S006</td><td>Ng</td><td>MAT101</td><td>Kalkulus I</td></tr><tr><td></td><td>S007</td><td>Aisyah</td><td>MAT101</td><td>Kalkulus I</td></tr><tr><td></td><td>S008</td><td>Yoges</td><td>BUS101</td><td>Prinsip Perniagaan</td></tr><tr><td></td><td>S009</td><td>Elijah</td><td>ENG101</td><td>Mekanikal Kejuruteraan</td></tr></table>		StudentID	SFirstName	CourseID	CourseName	▶	S001	Tan	FRN101	Bahasa Perancis I		S002	Nadia	FRN101	Bahasa Perancis I		S003	Wei	FRN101	Bahasa Perancis I		S004	Chau	CSC201	Sistem Pangkalan Data		S005	Hafiz	CSC201	Sistem Pangkalan Data		S006	Ng	MAT101	Kalkulus I		S007	Aisyah	MAT101	Kalkulus I		S008	Yoges	BUS101	Prinsip Perniagaan		S009	Elijah	ENG101	Mekanikal Kejuruteraan
	StudentID	SFirstName	CourseID	CourseName																																															
▶	S001	Tan	FRN101	Bahasa Perancis I																																															
	S002	Nadia	FRN101	Bahasa Perancis I																																															
	S003	Wei	FRN101	Bahasa Perancis I																																															
	S004	Chau	CSC201	Sistem Pangkalan Data																																															
	S005	Hafiz	CSC201	Sistem Pangkalan Data																																															
	S006	Ng	MAT101	Kalkulus I																																															
	S007	Aisyah	MAT101	Kalkulus I																																															
	S008	Yoges	BUS101	Prinsip Perniagaan																																															
	S009	Elijah	ENG101	Mekanikal Kejuruteraan																																															

Code	Output																																												
Left Outer Join Lists all courses and the number of students enrolled in each, including courses with zero students, by using LEFT JOIN to preserve all rows from the Courses table <pre>SELECT c.CourseID, c.CourseName, COUNT(e.StudentID) AS TotalStudents FROM Courses c LEFT JOIN Enrollments e ON c.CourseID = e.CourseID GROUP BY c.CourseID;</pre>	Result Grid   Filter Rows: Exp <table><thead><tr><th></th><th>CourseID</th><th>CourseName</th><th>TotalStudents</th></tr></thead><tbody><tr><td>▶</td><td>BUS101</td><td>Prinsip Perniagaan</td><td>1</td></tr><tr><td></td><td>BUS201</td><td>Asas Pemasaran</td><td>0</td></tr><tr><td></td><td>CSC101</td><td>Asas Pengaturcaraan</td><td>0</td></tr><tr><td></td><td>CSC201</td><td>Sistem Pangkalan Data</td><td>2</td></tr><tr><td></td><td>ENG101</td><td>Mekanikal Kejuruteraan</td><td>1</td></tr><tr><td></td><td>ENG201</td><td>Termodinamik</td><td>0</td></tr><tr><td></td><td>FRN101</td><td>Bahasa Perancis I</td><td>3</td></tr><tr><td></td><td>FRN201</td><td>Bahasa Perancis II</td><td>0</td></tr><tr><td></td><td>MAT101</td><td>Kalkulus I</td><td>2</td></tr><tr><td></td><td>MAT201</td><td>Algebra Linear</td><td>0</td></tr></tbody></table>		CourseID	CourseName	TotalStudents	▶	BUS101	Prinsip Perniagaan	1		BUS201	Asas Pemasaran	0		CSC101	Asas Pengaturcaraan	0		CSC201	Sistem Pangkalan Data	2		ENG101	Mekanikal Kejuruteraan	1		ENG201	Termodinamik	0		FRN101	Bahasa Perancis I	3		FRN201	Bahasa Perancis II	0		MAT101	Kalkulus I	2		MAT201	Algebra Linear	0
	CourseID	CourseName	TotalStudents																																										
▶	BUS101	Prinsip Perniagaan	1																																										
	BUS201	Asas Pemasaran	0																																										
	CSC101	Asas Pengaturcaraan	0																																										
	CSC201	Sistem Pangkalan Data	2																																										
	ENG101	Mekanikal Kejuruteraan	1																																										
	ENG201	Termodinamik	0																																										
	FRN101	Bahasa Perancis I	3																																										
	FRN201	Bahasa Perancis II	0																																										
	MAT101	Kalkulus I	2																																										
	MAT201	Algebra Linear	0																																										

Code	Output																												
Self Join Pairs students who joined in the same enrollment year by joining the Students table with itself and ensuring no duplicate or reversed pairs using a < comparison on StudentID <pre>SELECT s1.StudentID AS Student1, s2.StudentID AS Student2, s1.EnrollmentYear FROM Students s1 JOIN Students s2 ON s1.EnrollmentYear = s2.EnrollmentYear AND s1.StudentID < s2.StudentID;</pre>	<table><tr><th></th><th>Student1</th><th>Student2</th><th>EnrollmentYear</th></tr><tr><td>▶</td><td>S001</td><td>S005</td><td>2023</td></tr><tr><td></td><td>S001</td><td>S007</td><td>2023</td></tr><tr><td></td><td>S002</td><td>S006</td><td>2022</td></tr><tr><td></td><td>S003</td><td>S008</td><td>2024</td></tr><tr><td></td><td>S004</td><td>S009</td><td>2021</td></tr><tr><td></td><td>S005</td><td>S007</td><td>2023</td></tr></table>		Student1	Student2	EnrollmentYear	▶	S001	S005	2023		S001	S007	2023		S002	S006	2022		S003	S008	2024		S004	S009	2021		S005	S007	2023
	Student1	Student2	EnrollmentYear																										
▶	S001	S005	2023																										
	S001	S007	2023																										
	S002	S006	2022																										
	S003	S008	2024																										
	S004	S009	2021																										
	S005	S007	2023																										

D. Indexing and Optimization

- Create **two indexes**:
 - **BTREE** (for numeric/date columns).
 - **TEXT** (for text search).

Code

BTREE- Create Enrollments (EnrollmentDate)

```

1 • CREATE INDEX idx_enrollmentdate
2   ON Enrollments (EnrollmentDate);

SELECT *
FROM Enrollments
WHERE EnrollmentDate BETWEEN '2025-02-03' AND '2025-03-02';

```

Output

	EnrollmentID	StudentID	CourseID	EnrollmentDate	Status
▶	7	S007	MAT101	2025-02-03	Enrolled
*	NULL	NULL	NULL	NULL	NULL

FULLTEXT INDEX - NAME SEARCHING

```

1 • ALTER TABLE Students
2   ADD FULLTEXT idx_student_name (SFirstName, SLastName);

```

```

1 • SELECT *
FROM Students
2   WHERE MATCH(SFirstName, SLastName) AGAINST ('Aisyah');

```

	StudentID	SFirstName	SLastName	SPhone	EnrollmentYear	Status	SEmail
▶	S007	Aisyah	Nabila	012-7777777	2023	Active	aisyahnabila@graduate.utm.my
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Analyze query plans before and after index creation.

BTREE INDEX

Code	Output																										
Before index creation <pre>1 • EXPLAIN SELECT * 2 FROM Enrollments 3 WHERE EnrollmentDate BETWEEN '2025-02-03' AND '2025-03-02'; -</pre>	<table><tr><th></th><th>id</th><th>select_type</th><th>table</th><th>partitions</th><th>type</th><th>possible_keys</th><th>key</th><th>key_len</th><th>ref</th><th>rows</th><th>filtered</th><th>Extra</th></tr><tr><td>▶</td><td>1</td><td>SIMPLE</td><td>Enrollments</td><td>NULL</td><td>ALL</td><td>NULL</td><td>NULL</td><td>NULL</td><td>NULL</td><td>9</td><td>11.11</td><td>Using where</td></tr></table>		id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra	▶	1	SIMPLE	Enrollments	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra															
▶	1	SIMPLE	Enrollments	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where															
After index creation <pre>1 • EXPLAIN SELECT * 2 FROM Enrollments FORCE INDEX(idx_enrollment_date) 3 WHERE EnrollmentDate BETWEEN '2025-02-03' AND '2025-03-02'; -</pre>	<table><tr><th></th><th>id</th><th>select_type</th><th>table</th><th>partitions</th><th>type</th><th>possible_keys</th><th>key</th><th>key_len</th><th>ref</th><th>rows</th><th>filtered</th><th>Extra</th></tr><tr><td>▶</td><td>1</td><td>SIMPLE</td><td>Enrollments</td><td>NULL</td><td>range</td><td>idx_enrollment_date</td><td>idx_enrollment_date</td><td>4</td><td>NULL</td><td>1</td><td>100.00</td><td>Using index condition</td></tr></table>		id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra	▶	1	SIMPLE	Enrollments	NULL	range	idx_enrollment_date	idx_enrollment_date	4	NULL	1	100.00	Using index condition
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra															
▶	1	SIMPLE	Enrollments	NULL	range	idx_enrollment_date	idx_enrollment_date	4	NULL	1	100.00	Using index condition															

FULLTEXT INDEX

Code

Output

Before index creation

```

1 • EXPLAIN SELECT *
2   FROM Students
3   WHERE SFirstName LIKE '%Aisyah%'
4     OR SLastName LIKE '%Aisyah%';

```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Students	NULL	ALL	NULL	NULL	NULL	NULL	10	20.99	Using where

After index creation

```

1 • EXPLAIN SELECT *
2   FROM Students
3   WHERE MATCH(SFirstName, SLastName) AGAINST ('Aisyah');

```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Students	NULL	fulltext	idx_student_name	idx_student_name	0	const	1	100.00	Using where; Ft_hints: sorted

- Write a short interpretation of what changed (rows examined, possible keys).

INDEX	rows examined	possible keys
BTREE index	Before: 9 After: 1	Before: null After: idx_enrollment_date
FULLTEXT index	Before:10 After: 1	Before: null After: idx_student_name

Conclusion

In order to meet the requirements of a modernized academic attendance solution, this project successfully developed the core SQL-based backend for a digital attendance management system for universities and schools. By substituting a structured and centralized database for traditional paper-based methods, the system guarantees faster, more accurate, and more accessible attendance tracking. The database supports essential entities like students, lecturers, courses, enrollments, sessions, and attendance records through carefully designed tables and relationships.

A full suite of SQL operations, including DDL for database creation, DML for data management, and DQL for advanced data retrieval, were implemented by the team during the development phase. In order to show a fully functional backend, the team also used constraints to preserve data integrity and carried out operations like filtering, aggregation, joins, subqueries, and conditional logic. Additionally, to improve query performance, indexing strategies utilizing the BTREE and TEXT indexes were combined, and an analysis of the results before and after indexing was accomplished to demonstrate efficiency gains.

As each member contributed to various aspects of the backend development, this project not only improved the group's SQL proficiency but also strengthened teamwork. After identifying issues like communication and task distribution, the team recommended further talks to enhance teamwork in subsequent projects. The project's objectives were generally met by providing a comprehensive database system, full SQL operations, optimization strategies, and understandable documentation, which established a solid basis for integration with actual front-end applications.

Team Work Conclusion

Name	Role & Learning	Challenges & Suggestion
CHAU YING JIA	I started up the project, choosing the topic and brainstorming the required tables so that my team could proceed smoothly. I also drafted the introduction to ensure we understood the system's purpose and objectives.	Challenge: The main challenge I faced was dividing tasks fairly. I also needed to review my teammates' work after they finished so I could learn beyond the DDL section. Suggestion: If SQL Workbench has a feature on collaborative editing, it is easier for us to work together.
TAN ZHI MING	I handled the DML tasks by inserting and preparing all required data for the database. I also worked on key DQL queries, such as retrieving active students and filtering enrollment by specific periods. Additionally, I reviewed the outputs to ensure the data and queries were accurate and consistent.	Challenge: Merging all team members' database code was difficult because we had no version-control tool to detect conflicts or ensure safe integration. Differences in schema structure also caused errors during merging. Suggestion: Use a proper version-control workflow and enforce a consistent schema to avoid merging conflicts in future work.
ELIJAH SHE YU SHENG	Contribute the part which are BTREE index and text index, learn more how to sort the data efficiently.	Challenge: distribution of task Suggestion: having more discussion so that we can solve the problem as fast as possible.
NG YU HIN	I ensured retrieved data was accurate, consistent, and aligned with project requirements. I also combine multiple tables using different types of JOINS. How set operations like UNION, and NOT EXISTS behave with different datasets and learned conditional logic directly inside SQL queries	Challenge : Queries returning no results Suggestion : Verify and test data consistency and matching keys before writing queries

Presentation Video URL

<https://youtu.be/a-kKAvEAQF4>



UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING
2025/2026 - SEM 1

Project 1 - Smart Digital Attendance System

Name	:	CHAU YING JIA
Matric Id	:	A23CS0213
Name	:	TAN ZHI MING
Matric Id	:	A23CS0189
Matric Id	:	ELIJAH SHE YU SHENG
Name	:	A23CS0073
Matric Id	:	NG YU HIN
Name	:	A23CS0148



Innovating Solutions

Marking Rubric

SQL Implementation (CLO1)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Backend SQL Implementation (DDL/DML/Queries)	All SQL statements execute flawlessly; covers full range – DDL, DML, SELECT, joins, subqueries, functions.	Most SQL correct; small syntax or logic issues.	Several working statements but limited variety.	Few working statements; repetitive or basic logic.	Non-functional / incomplete SQL.	
Advanced Data Logic (Filtering, Aggregation, Subquery)	Uses complex logic (nested queries, GROUP BY, HAVING, CASE, conditions) effectively.	Correct use of aggregations with minor gaps.	Some queries too simple or misapplied.	Few aggregations or logical errors.	Not attempted.	
Indexing & Optimization	Correctly creates BTREE and TEXT indexes; shows analysis before/after; interprets performance gain.	Indexes implemented; partial analysis.	Indexing attempted without analysis.	Incomplete index usage.	Not attempted.	
Transaction Control & Front-End Logic Integration	Demonstrates atomic operations (START TRANSACTION, COMMIT, ROLLBACK); explains how it fits system workflow (front-end use case e.g. booking/purchase).	Transaction logic corrects but limited integration explanation.	Partial transaction implementation.	Minimal attempt, unclear scenario.	Missing transaction logic.	
Subtotal						

Teamwork and Collaboration (CLO3)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Team Coordination & Contribution	Clear task division; balanced participation; strong collaboration evident in report and presentation; each member contributes to SQL work.	Good teamwork and participation with minor imbalance.	Some collaboration; limited task clarity.	Minimal teamwork; dominated by few members.	No teamwork evidence; incomplete group submission.	
Subtotal						

Documentation & Presentation (CLO4)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Report Documentation	Clear and complete report including intro, SQL summary, query screenshots, optimization, transaction explanation, and reflection.	Report complete with minor formatting or clarity issues.	Adequate report but lacks details or screenshots.	Weakly structured or incomplete sections.	Missing or very poor documentation.	
Slide Quality & Visual Communication	Slides are professional, organised, visually clear, and consistent with project flow.	Well-prepared slides with minor design issues.	Slides understandable but inconsistent layout.	Poorly structured or cluttered slides.	Missing or unreadable slides.	
Verbal Presentation	Excellent delivery, confident speakers, balanced time usage, fluent explanation, and effective Q&A handling.	Good delivery; some coordination issues.	Understandable but low energy; moderate Q&A.	Disorganized delivery or weak explanations.	Unable to present or answer questions.	
Subtotal						